CSE221: Algorithms

D & C Based Sorting Algorithms

Prepared by: A B M Muntasir Rahman [MUNR]



Divide & Conquer Approach

Divide	**the problem (instance) into subproblems.**
Conquer	**the subproblems by solving them recursively.**
Combine	**the solutions to subproblems.**

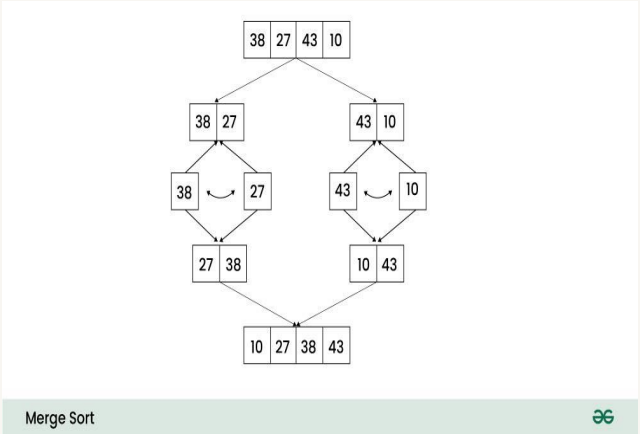
Examples

- Binary/Ternary Search
- Merge Sort
- Quick Sort
- Maximum Sum Subarray
- Karatsuba's Algorithm for Multiplication etc.

Time Complexity: aT(n/b) + f(n)

Merge Sort

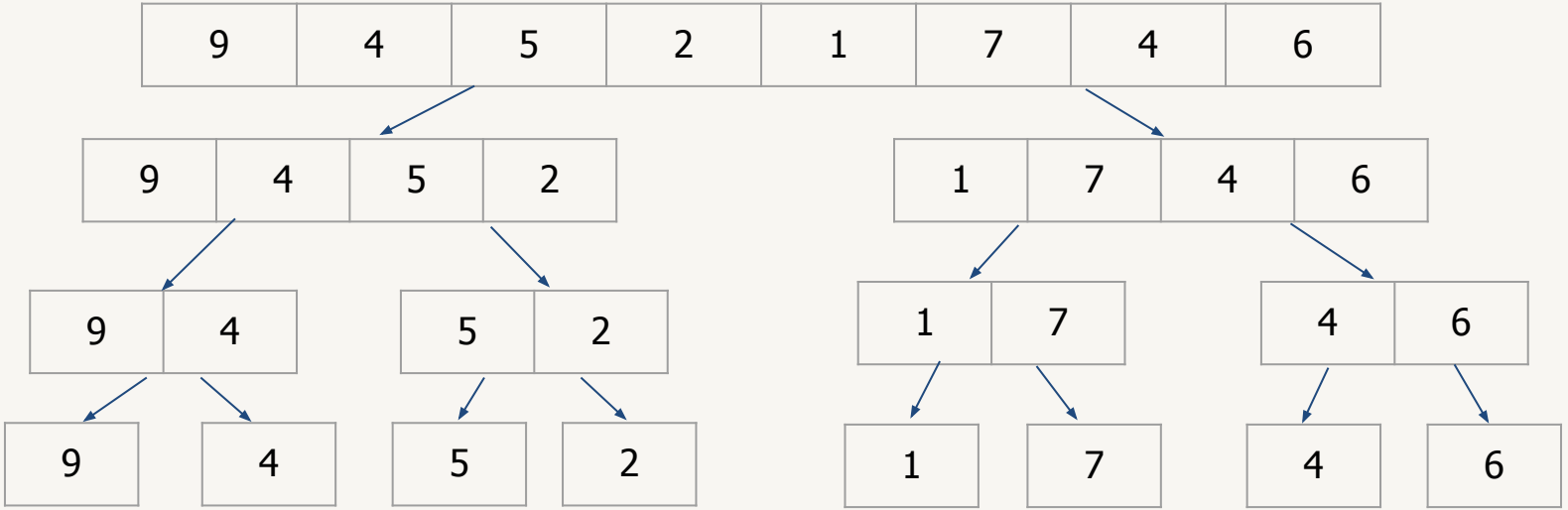
- A divide & conquer sorting algorithm
- Recursively divides the array into smaller subarrays and then by sorting these subarrays while merging obtains the sorted array.



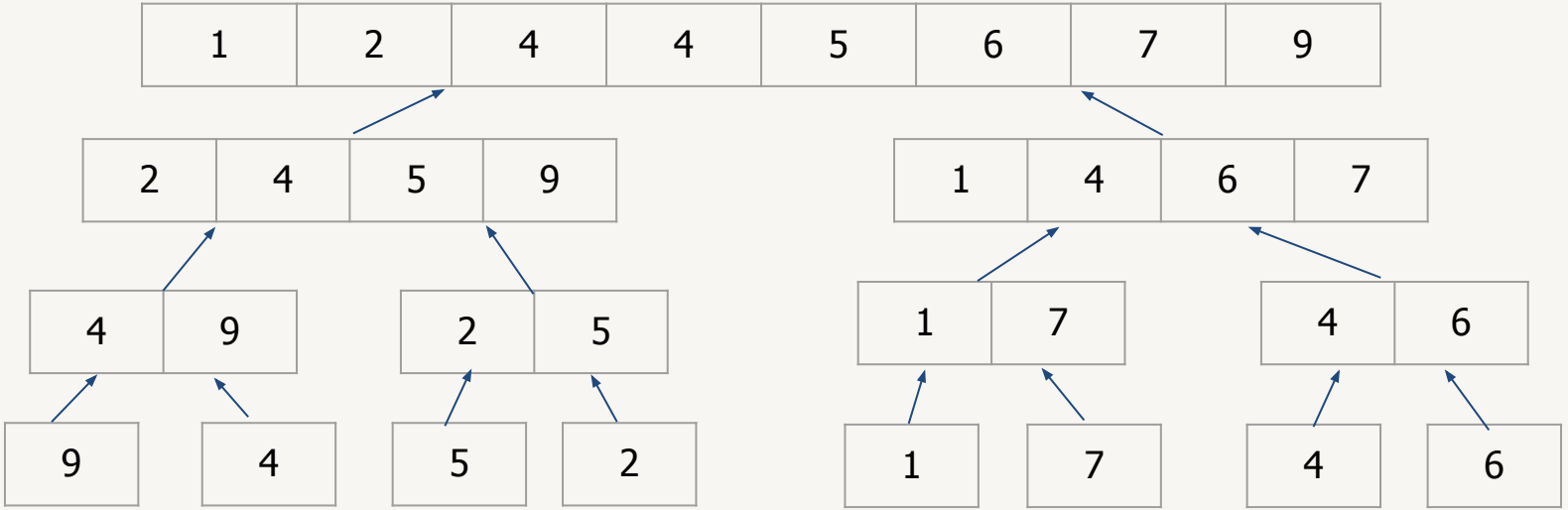
How does Merge Sort work?

- **Divide:** Divide the array or dataset recursively into two halves until it can no more be divided.
- **Conquer:** Each smallest subarray is automatically sorted individually using the merge sort algorithm.
- **Merge:** The sorted subarrays are merged back together in sorted order and the process continues until all elements from both subarrays have been merged.

Merge Sort (Simulation)



Merge Sort (Simulation)



Merge Sort (Pseudocode)

```
mergeSort(arr)
    if (len(arr) <= 1)
        return arr
    mid = len(arr) // 2
    left = mergeSort(arr[:mid])
    right = mergeSort(arr[mid:])
    return merge(left, right)
```

Merge Sort (Pseudocode)

```
merge(left, right)
    a = arrayOf(len(left) + len(right))
    i, j, k = 0, 0, 0
    while (i < len(left) && j < len(right))
        if (left[i] <= right[j])
            a[k] = left[i]
            i = i + 1
        else
            a[k] = right[j]
            j = j + 1
        k = k + 1
    while (i < len(left))
        a[k] = left[i]
        i = i + 1
        k = k + 1
    while (j < len(right))
        a[k] = right[j]
        j = j + 1
        k = k + 1

    return a
```

Time Complexity Analysis

```
mergeSort(arr)                          → T(n)
    if (len(arr) <= 1)
        return arr
    mid = len(arr) // 2                 → 1
    left = mergeSort(arr[:mid])         → T(n/2)
    right = mergeSort(arr[mid:])        → T(n/2)
    return merge(left, right)           → n
```

Time Complexity Analysis

∴ **T(n) = 2T(n/2) + n**

Solving with tree/substitution/master method,

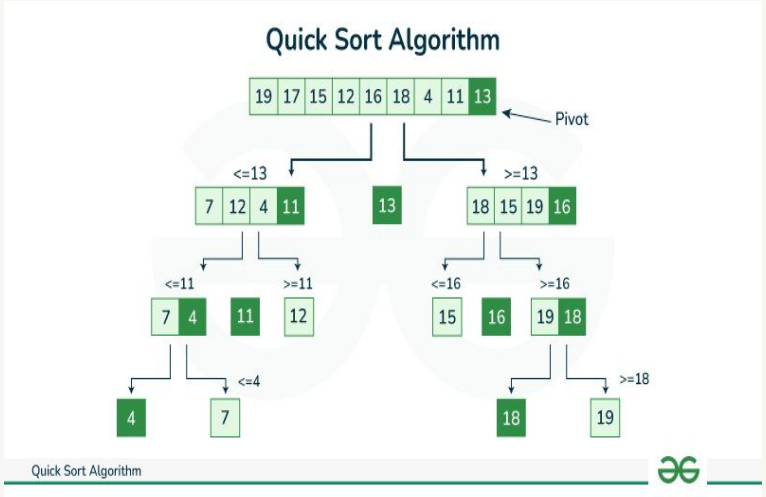
T(n) = O(nlogn)

Merge Sort (Properties)

- **Time Complexity:**
  - Best/Average/Worst Case: O(nlogn).
- **Space Complexity:**
  - Best/Average/Worst case: O(n).
- **Stable:** Yes.
- **In-place:** No.
- Suitable for larger arrays. Unsuitable for smaller arrays and requires additional memory store the merged subarrays.

Quicksort

- A divide and conquer sorting algorithm.
- Picks a pivot and partitions the input array around the picked pivot by placing it in it's correct position in sorted array.



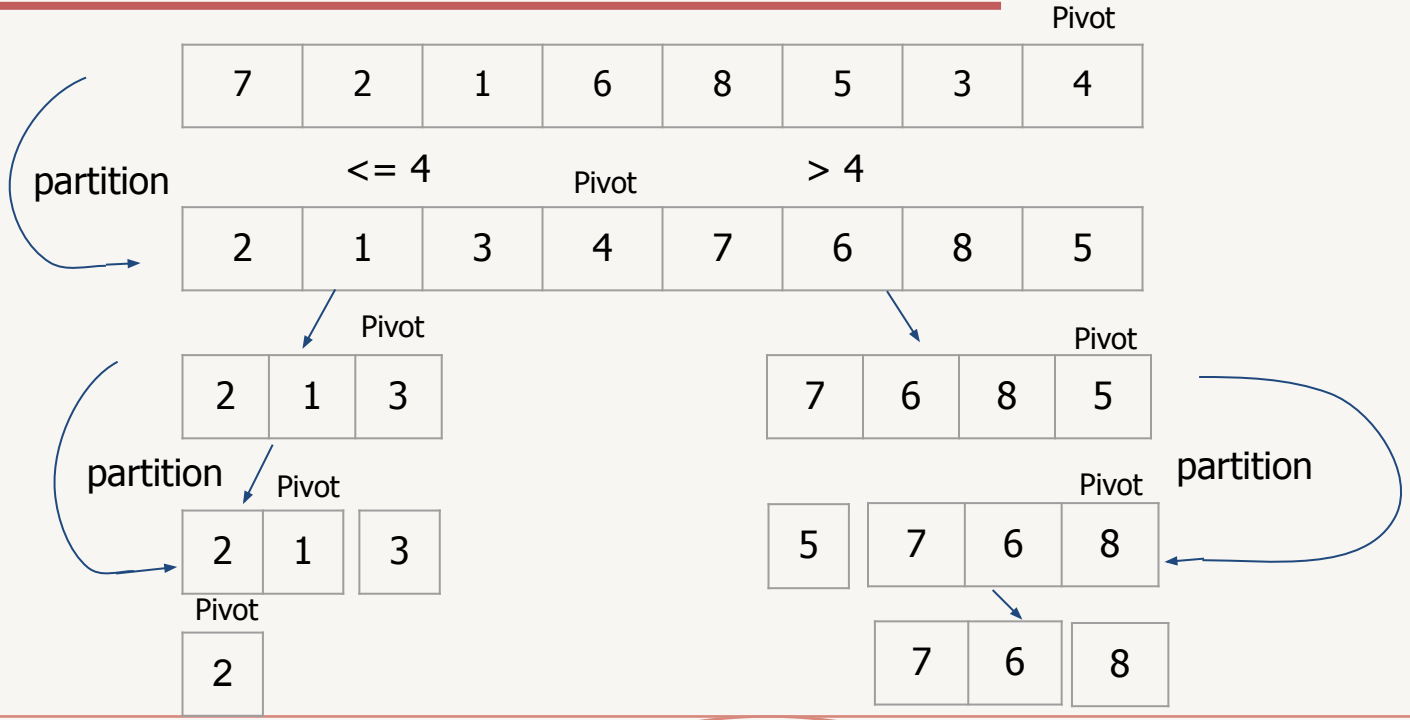
How does Quicksort work?

- **Divide:** The array or dataset is divide into subarrays by selecting a pivot element. While dividing, the pivot should be positioned in such a way that all elements less than pivot are on the left side and elements greater than the pivot are on the right side of the pivot. This process continues until each subarray contains a single element.

- **Conquer:** Each smallest subarray is automatically sorted at this point.

- **Combine:** All subarrays are combined to form a sorted array.

Quicksort (Simulation)



Quicksort (Pseudocode)

```
quickSort(arr, start, end){
    if (start >= end){
        return arr
    }
    pIndex = partition(arr, start, end)
    quickSort(arr, start, pIndex - 1)
    quickSort(arr, pIndex + 1, end)
    return arr
}
```

```
partition(arr, start, end){
    pivot = arr[end]
    pIndex = start
    for(int i = start, i <= end-1; i++){
        if (arr[i] <= pivot){
            swap(arr[i], arr[pIndex])
            pIndex = pIndex + 1
        }
    }
    swap(arr[pIndex], arr[end])
    return pIndex
}
```

Time Complexity Analysis

```
quickSort(arr, start, end){
    if (start >= end){
        return arr
    }
    pIndex = partition(arr, start, end)
    quickSort(arr, start, pIndex - 1)
    quickSort(arr, pIndex + 1, end)
    return arr
}
```

T(n)

n

T(n/2)

T(n/2)

Time Complexity Analysis

∴ **T(n) = 2T(n/2) + n**

Solving with tree/substitution/master method,

T(n) = O(nlogn)

Worst Case

1	2	3	4	5	6	7	8

- For above sorted array, if we choose first element as pivot, there will be no left subarray. Again, if we choose last element as pivot, there will be no right subarray.
- In such cases, the recurrence relation becomes T(n) = T(n - 1) + n. **Which, by solving, we get $O(n^2)$ in worst case.**
- However, this can be easily avoided by choosing mid or other element as pivot.

Quick Sort (Properties)

- **Time Complexity:**
  - Best/Average Case: O(nlogn).
  - Worst Case: $O(n^2)$.
- **Space Complexity:**
  - Best/Average Case: O(log n).
  - Worst Case: O(n).
- **Stable:** No.
- **In-place:** Yes (in best/average case).
- Suitable for larger arrays. Unsuitable for smaller arrays and is not a stable sorting algorithm.